

Assignment for Introduction to Java - 1

Q1- Write a class with FirstName, LastName & age field. Print Firstname, LastName & age using static block, static method & static variable respectively.

```
// TIP: To <b>run</b> code, press <shortcut actionId="Run"/> or  
// click the <icon src="AllIcons.Actions.Execute"/> icon in the gutter.  
public class Main {  
    public static void main(String[] args) {  
        Student student1 = new Student();  
  
        System.out.println("Using Static variable");  
        System.out.println(Student.age);  
        Student.displayData();  
    }  
}
```

```
public class Student { 4 usages  
    static String firstName; 3 usages  
    static String lastName; 3 usages  
    static int age; 4 usages  
  
    static {  
        System.out.println("Using Static block");  
        firstName = "Aman";  
        lastName = "Pal";  
        age = 23;  
        System.out.println(firstName + " " + " " + lastName + " " + age);  
    }  
  
    static void displayData(){ 1 usage  
        System.out.println("Using Static method");  
        System.out.println(firstName + " " + " " + lastName + " " + age);  
    }  
}
```

Q2- Write a program to read user input until user writes XDONE and then show the entered text by the user on command line.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        StringBuilder input = new StringBuilder();
        System.out.println("Enter your text");
        while(true){
            String line = sc.nextLine();
            if(line.contains("XDONE")){
                break;
            }
            input.append(line);
        }
        sc.close();
        System.out.println(input);
    }
}
```

Main x

```
/home/aman/.jdk/openjdk-24.0.1/bin/java -javaagent:/s
Enter your text
this
is
XDONE
this is

Process finished with exit code 0
```

Q3- Write a java program to show following menu to the user:

*******Menu*******

- 1. Calculate Area of Circle**
- 2. Calculate Circumference of a Circle**
- 3. Exit.**

Choose an option (1-3):

Take radius as user input.

Hint: Use Switch statement to act on the menu. Also area and circumference methods should be static

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("*****Menu*****\n1. Calculate Area of Circle\n"
+
                "2. Calculate Circumference of a Circle\n" +
                "3. Exit.\n" +
                "Choose an option (1-3): ");
        int option = sc.nextInt();
        switch (option) {
            case 1: {
                System.out.println("Enter radius");
                double radius = sc.nextDouble();
                System.out.println(Calculate.circleArea(radius));
                break;
            }
            case 2: {
                System.out.println("Enter radius");
                double radius = sc.nextDouble();
                System.out.println(Calculate.circleCircumference(radius));
                break;
            }
            case 3: {
                System.out.println("Exiting...");
                break;
            }
            default: {
                System.out.println("Please select a valid option");
            }
        }
    }
}
```

Q4- Create a two dimensional array of integers and display: 1. sum of all elements of each column 2. sum of all elements of each row

```
public class Main {  
    public static void main(String[] args) {  
        int[][] arr = {  
            {1,2,3},  
            {4,5,6},  
            {7,8,9}  
        };  
        int rows = arr.length;  
        int cols = arr[0].length;  
        System.out.println("Sum of each row:");  
        for (int i = 0; i < rows; i++) {  
            int rowSum = 0;  
            for (int j = 0; j < cols; j++) {  
                rowSum += arr[i][j];  
            }  
            System.out.println(i + " " + rowSum);  
        }  
  
        System.out.println("\nSum of each column:");  
        for (int j = 0; j < cols; j++) {  
            int colSum = 0;  
            for (int i = 0; i < rows; i++) {  
                colSum += arr[i][j];  
            }  
            System.out.println(j+ " " + colSum);  
        }  
    }  
}
```

Q5- Create a class named Employee with fields firstname,lastname,age and designation. The class should:

- 1. have all types of constructors to initialize the object**
- 2. class should also have setter methods to update a particular field**
- 3. Override its toString method to display a meaningful message using all these fields.**

```
public class Employee {
    private String firstName;
    private String lastName;
    private int age;
    private String designation;

    public Employee() {
        this.firstName = "Unknown";
        this.lastName = "Unknown";
        this.age = 0;
        this.designation = "Not Assigned";
    }

    public Employee(String firstName, String lastName, int age, String
designation) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.designation = designation;
    }

    public Employee(Employee other) {
        this.firstName = other.firstName;
        this.lastName = other.lastName;
        this.age = other.age;
        this.designation = other.designation;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public String getFirstName() {
        return firstName;
    }
}
```

```
public void setFirstName(String firstName) {
    this.firstName = firstName;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}

public String getDesignation() {
    return designation;
}

public void setDesignation(String designation) {
    this.designation = designation;
}

public String toString() {
    return firstName + " " + lastName + " " + age + " " +
designation;
}
}
```

Q6- Implement a banking system using java. Create 3 sub class of Bank : SBI, BOI, ICICI
Classes should have attributes like Name, headofficeAddress, chairmanName, branchCount, fdInterestRate, personalLoanInterestRate, homeLoanInterestRate. All 3 should have following methods:

1. add getters and setters for the fields
2. print details of every bank (override toString)

Base Class:

```
public class Bank {  
    private String name;  
    private String headOfficeAddress;  
    private String chairmanName;  
    private int branchCount;  
    private double fdInterestRate;  
    private double personalLoanInterestRate;  
    private double homeLoanInterestRate;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getHeadOfficeAddress() {  
        return headOfficeAddress;  
    }  
  
    public void setHeadOfficeAddress(String headOfficeAddress) {  
        this.headOfficeAddress = headOfficeAddress;  
    }  
  
    public String getChairmanName() {  
        return chairmanName;  
    }  
  
    public void setChairmanName(String chairmanName) {  
        this.chairmanName = chairmanName;  
    }  
  
    public int getBranchCount() {  
        return branchCount;  
    }  
}
```

```

    public void setBranchCount(int branchCount) {
        this.branchCount = branchCount;
    }

    public double getPersonalLoanInterestRate() {
        return personalLoanInterestRate;
    }

    public void setPersonalLoanInterestRate(double
personalLoanInterestRate) {
        this.personalLoanInterestRate = personalLoanInterestRate;
    }

    public double getFdInterestRate() {
        return fdInterestRate;
    }

    public void setFdInterestRate(double fdInterestRate) {
        this.fdInterestRate = fdInterestRate;
    }

    public double getHomeLoanInterestRate() {
        return homeLoanInterestRate;
    }

    public void setHomeLoanInterestRate(double homeLoanInterestRate) {
        this.homeLoanInterestRate = homeLoanInterestRate;
    }

    public String toString() {
        return getName() + getHeadOfficeAddress() + getChairmanName()
+ getBranchCount() + getFdInterestRate() +
getPersonalLoanInterestRate() + getHomeLoanInterestRate();
    }
}

```

Example child class having all methods

```

public class SBI extends Bank {
    public SBI() {
        setName("SBI");
        setHeadOfficeAddress("Mumbai, India");
        setChairmanName("Dinesh Kumar");
    }
}

```

```

        setBranchCount(220);
        setFdInterestRate(6.5);
        setPersonalLoanInterestRate(11);
        setHomeLoanInterestRate(15);
    }
}

```

Q7- WAP showing try, multi-catch and finally blocks.

```

public class Main {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;

            int result = a / b;

            System.out.println("Ans" + result);
        }
        catch (ArithmeticException | NullPointerException e) {
            System.out.println("An exception occurred: " + e);
        }
        finally {
            System.out.println("This will always execute");
        }

        System.out.println("End.");
    }
}

```

Q8- WAP to produce NoClassDefFoundError and ClassNotFoundException exception.

For NoClassDefFoundError

you have two classes: Test and Helper.

1. Compile both of them.
2. Delete Helper.class
3. Now run Test, which tries to create an object of Helper.

4. JVM cannot find `Helper.class`, so it throws `NoClassDefFoundError`.

For ClassNotFoundException

□ Main ×

```
/home/aman/.jdk/openjdk-24.0.1/bin/java -javaagent:/snap/intellij-idea-24.1.4/bin/idea-agent.jar -cp /home/aman/.m2/repository/org/abc/ClassNotFoundException/1.0.0/ClassNotFoundException-1.0.0.jar ClassNotFoundExceptionjava.lang.ClassNotFoundException: ABC

Process finished with exit code 0
```

Q9- Create a custom exception that do not have any stack trace.

Custom class extending exception class

```
class MyException extends Exception {  
  
    public MyException(String message) {  
  
        super(message);  
  
    }  
  
    public Throwable fillInStackTrace() {  
  
        return this;  
  
    }  
  
}
```

Main funation

```
public class Main {  
  
    public static void main(String[] args) {  
  
        try {  
  
            throw new MyException("Custom error without stack trace");  
  
        } catch (MyException e) {  
  
            System.out.println("Caught: " + e);  
  
            e.printStackTrace();  
  
        }  
  
    }  
  
}
```

